ОТЧЁТ
ПО ДОМАШНЕЙ РАБОТЕ № 4

«Обеспечение отказоустойчивости средствами Apache ZooKeeper»

Индивидуальный вариант № 11 - DevOps-инженер

Выполнил: Алёшкин Дмитрий Игоревич
Группа: J3311
ИСУ: 464975

2026

1. Цель и постановка задачи

Цель работы - исследовать отказоустойчивость сервиса координации Apache ZooKeeper и реализовать переключение между двумя экземплярами координатора вычислений при потере активности действующего лидера. Отдельная цель - показать проблему устаревшего владельца (stale writer) и проверить механизм защиты от его записей с использованием версии и эпохи.

Основой вычислений является предыдущая домашняя работа № 3: анализ DevOps-вакансий, где были получены эталонные частоты неупорядоченных пар навыков. Новая работа не заменяет этот расчёт имитацией, а повторяет фактическую обработку данных с возможностью продолжения после отказа.

Таблица 1 - Целевые показатели и контрольные результаты

Контрольный результат	Значение
Вакансий варианта № 11	5 158
Чанков обработки	21 (20 × 250 + 158)
Уникальных пар навыков	22 311
Суммарных вхождений пар	134 786
Проверка результата	Точное совпадение частоты каждой пары с ДЗ № 3

Проверяемые требования: конфигурация трёхсерверного ансамбля и кворума, эфемерный узел, переключение лидера между координаторами, восстановление с чекпоинта, две публикации без защиты, отклонение устаревшей записи с FENCING, измерение времени failover и сохранение результатов опытов.

2. Испытательный стенд и инфраструктура

Таблица 2 - Конфигурация испытательного стенда

Компонент	Фактическая конфигурация
Операционная система	Windows 11
Контейнеризация	Docker Desktop; Docker Compose
Сервис координации	Apache ZooKeeper 3.9.6, Java 17
Узлы ансамбля	lab4-zk1 / lab4-zk2 / lab4-zk3
Опубликованные клиентские порты	127.0.0.1:2181, 2182, 2183
Координаторы	Python 3.11, Kazoo 2.10.0, worker-a / worker-b
Сессия координаторов	10 секунд

Компонент	Фактическая конфигурация
Рабочий каталог	C:\rustfs-lab\devops-hw\lab4
Данные ДЗ № 3	CSV с 5 158 DevOps-вакансиями; эталон пар skills_2.csv

Ансамбль развёрнут отдельным Docker Compose-проектом. Каждый ZooKeeper-сервер имеет собственный идентификатор и именованный том для данных. Участники обмениваются служебными сообщениями по внутренней сети Docker, а наружу выведены три независимых клиентских порта.

Все три ZooKeeper-сервера физически размещены на одном компьютере. Поэтому эксперимент демонстрирует логические отказы отдельных контейнеров и сохранение кворума, но не независимость от отказа всего хоста.

3. Архитектура решения



Рисунок 1 - Логическая архитектура ансамбля ZooKeeper и двух координаторов

Таблица 3 - Назначение основных компонентов

Компонент	Функция
ZooKeeper ensemble	Выбор и фиксация текущего лидера через ephemeral-sequential znodes
znode /election	Набор заявок координаторов на лидерство
znode /epoch	Монотонно возрастающее значение эпохи с условным обновлением
znode /pointer	Указатель на опубликованный результат с версией, эпохой и владельцем
Координатор A	Исходный активный обработчик; принудительно приостанавливается
Координатор B	Резервный обработчик; принимает роль лидера после истечения сессии A
Общий чекпоинт	Файл с номером следующего чанка и накопленными частотами пар

При подключении координатор создаёт эфемерно-последовательный узел с префиксом candidate-. Активным считается экземпляр с минимальным порядковым номером действующего узла. После приостановки A его сессия истекает, эфемерный узел исчезает и B получает возможность стать лидером. При вступлении в роль эпоха увеличивается атомарным сравнением версии узла.

Результат каждого запуска хранится отдельно от исходных данных ДЗ № 3. Это защищает эталонные файлы и позволяет повторять сценарии без перезаписи доказательств предыдущих запусков.

4. Проверка кворума ансамбля

Сначала три контейнера были запущены одновременно. Диагностика ролей показала наличие одного лидера и двух последователей. Затем два последователя останавливались по одному, проверялась доступность согласованных операций клиента, после чего серверы возвращались в состав ансамбля.

Таблица 4 - Последовательная проверка доступности и кворума

Этап	Состав ансамбля	Реальный результат
Все узлы работают	zk1 + zk2 + zk3	zk1 - follower; zk2 - leader; zk3 - follower; чтение/запись успешны
Остановлен один последователь	2 из 3	Кворум сохранён; запись и чтение клиента успешны
Остановлены два последователя	1 из 3	Запись клиента не завершилась успешно; получен тайм-аут подключения
Узлы возвращены	3 из 3	zk1 - follower; zk2 - follower; zk3 - leader

После восстановления произошла повторная проверка ролей: снова присутствовали ровно один leader и два follower. Сам факт смены конкретного лидера в ходе опыта допустим - ZooKeeper автоматически согласует новую конфигурацию при наличии большинства.

Примечание к интерпретации: в протоколе потери двух участников клиентский Kazoo получил Connection time-out; часть диагностических сообщений также касалась недоступности имён остановленных контейнеров. Это согласуется с потерей кворума, однако сам по себе клиентский тайм-аут не следует трактовать как универсальный код ошибки ZooKeeper.

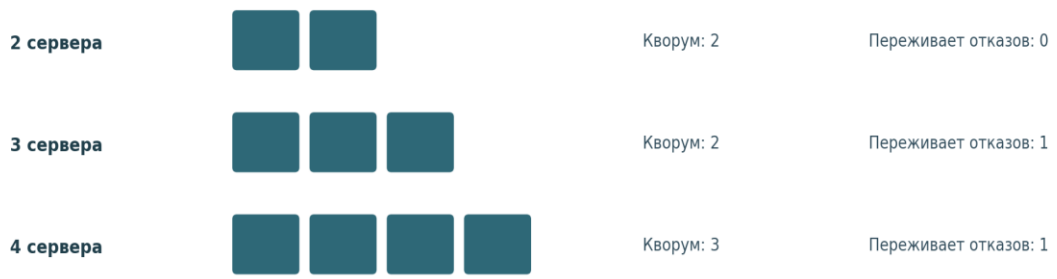


Рисунок 2 - Минимальные кворумы и число переживаемых отказов

Фрагмент итогового протокола; сведения из evidence/ensemble/quorum.json

```
Проверка кворума: все три - CLIENT_WRITE_READ_OK;
один остановлен - CLIENT_WRITE_READ_OK;
два остановлены - KazooTimeoutError / Connection time-out;
восстановление - 1 leader + 2 followers.
```

5. Проверка эфемерного узла

Эфемерный узел (ephemeral znode) существует, пока действует создавшая его клиентская сессия. В проведённом автоматическом тесте были открыты две независимые сессии клиента Kazoo. Первая создала ephemeral znode с уникальным именем, вторая подтвердила его существование. После завершения первой сессии вторая проверила, что узел исчез.

Таблица 5 - Демонстрация жизненного цикла ephemeral znode

Действие	Результат
Клиент-владелец создаёт ephemeral znode	Узел появился и доступен для чтения вторым клиентом
Закрытие сессии владельца	Сессия завершена через owner.stop()/close()
Проверка вторым клиентом	exists(path) вернул None; узел исчез без ручного delete

События из evidence/ensemble/ephemeral.txt; суффикс идентификатора опущен

```
EPHEMERAL_PRESENT /lab4/ephemeral-…
EPHEMERAL_DISAPPEARED_AFTER_OWNER_SESSION_CLOSE /lab4/ephemeral-…
```

Для учебной работы тест выполнен через клиент Kazoo, а не посредством двух интерактивных окон zkCli. Механизм и жизненный цикл znode проверены, но для полного соответствия формулировке «в zkCli» можно дополнительно повторить тот же сценарий в интерактивной консоли и приложить настоящий скриншот.

6. Исходные данные и система чекпоинтов

Расчёт продолжает работу с подготовленной на этапе ДЗ № 3 выборкой DevOps-вакансий. На вход координаторов передаются два доступных только для чтения файла: компактный набор vacancies с полями vacancy_id и key_skills и эталонный файл частот пар skills_2.csv. Размер выборки - 5 158 вакансий. SHA-256 подготовленного входа: 541174a6d1f5a80a949dfa7a4255934ea3659a6aa850c6e859892a5a6c2e9093.

Для каждой вакансии строка навыков разбивается символом «;», элементы очищаются от пробелов и дубли устраняются внутри вакансии. Затем навыки сортируются и для всех неупорядоченных пар увеличиваются частоты. Такой же способ формирования ключей использовался в ДЗ № 3, что делает поэлементную сверку осмысленной.

```
skills = sorted({s.strip() for s in key_skills.split(";") if s.strip()})
counts.update(itertools.combinations(skills, 2))
```

Вход делится на 21 логический чанк: 20 блоков по 250 вакансий и последний из 158. После обработки очередного чанка сохраняется checkpoint.json, содержащий число завершённых чанков, следующую позицию обработки и все накопленные частоты пар. При записи используются файловая блокировка и атомарная замена временного файла. Запасной координатор читает уже готовое состояние и обрабатывает только оставшуюся часть выборки.

Таблица 6 - Чекпоинты и контрольные величины вычисления

Показатель	Подтверждённое значение
Обработано вакансий	5 158
Всего блоков	21
Выполнено A до приостановки	5 блоков
Начальная позиция восстановления B	next_chunk = 5; первым заново рассчитывается блок № 6
Итоговое число уникальных пар	22 311
Итоговая сумма частот	134 786
Поэлементное сравнение	EXACT MATCH для всех 22 311 ключей

Файл чекпоинта хранится на общем локальном диске, примонтированном к обоим контейнерам, а не в ZooKeeper. Для учебного стенда этого достаточно, чтобы продемонстрировать сохранение прогресса, но такое решение не является распределённым устойчивым хранилищем для эксплуатации на независимых физических узлах.

7. Методика экспериментов с отказом координатора

Каждый запуск использует отдельный идентификатор опыта и отдельный znode-путь. Сначала запускается координатор A, получающий минимальный sequence-номер и эпоху 1. Затем к ансамблю подключается B и становится запасным. После записи пяти чекпоинтов сценарий вызывает docker pause для контейнера A. Клиент B наблюдает исчезновение эфемерной заявки A, принимает роль лидера и получает эпоху 2.

Для того чтобы B успел закончить расчёт, A остаётся приостановленным примерно 40 секунд. После docker unpause старый процесс продолжает работу со своим прежним состоянием, хотя прежняя сессия ZooKeeper уже недействительна. Различие двух экспериментов состоит исключительно в правилах приёма итоговой записи: без проверки версии либо с обязательным сравнением ожидаемой версии.

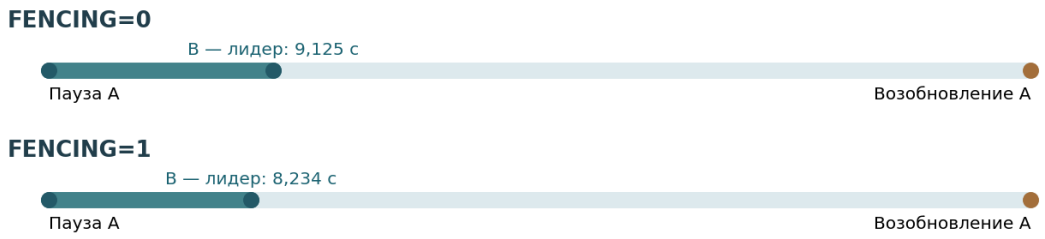


Рисунок 3 - Схема эксперимента: пауза A, избрание B и возобновление старого процесса

Время переключения измеряется на хосте Windows по монотонным часам: от момента команды приостановки A до обнаружения записи ROLE_LEADER от B в журнале. При опросе с периодом около 0,45 секунды результат включает задержку наблюдения и не претендует на микросекундную точность протокола.

8. Эксперимент без защиты: FENCING=0

Идентификатор итогового успешного опыта - no_fencing_20261026_070321. A был активным лидером с эпохой 1, сохранил пять чанков и приостановлен на ~40 секунд. Спустя 9,125 секунды запасной B зафиксирован как лидер с эпохой 2. Он восстановился после пятого чанка, выполнил оставшиеся вычисления, сверил результаты со всеми частотами эталона и опубликовал указатель версии 1.

После возобновления A также завершил расчёт и, поскольку проверка версии при публикации была отключена, безусловно обновил указатель. В журнале получились две успешные записи PUBLISHED_RESULT, а версия указателя

увеличилась до 2. Итоговым владельцем указателя оказался A с уже устаревшей эпохой 1.

Таблица 7 - Подтверждение повторной записи без FENCING

Событие	Зафиксированный результат
Пауза A	2026-10-26T07:03:27.489+00:00
Избрание B	9,125 с после паузы (host monotonic)
Продолжение B	Из чекпоинта после чанка 5
Публикация B	2026-10-26T07:03:39.245+00:00; версия 1; эпоха 2
Повторная публикация A	2026-10-26T07:04:17.739+00:00; версия 2; эпоха 1
Конечный указатель	owner=a; epoch=1; version=2
Контрольные частоты пар	EXACT MATCH: 22 311 ключей, 134 786 вхождений

Фрагмент подлинных событий журналов; порядок соответствует времени публикаций

```
{"worker":"b", "event":"PUBLISHED_RESULT", "epoch":2, "pointer_version":1}
{"worker":"a", "event":"PUBLISHED_RESULT", "epoch":1, "pointer_version":2}
```

Это подтверждает, что потеря ZooKeeper-лидерства сама по себе не останавливает запись старого процесса. Риск состоит не в ошибке подсчёта частот - они остались корректными, - а в некорректной смене владельца опубликованного результата.

9. Эксперимент с защитой: FENCING=1

Идентификатор итогового успешного опыта - fencing_20261026_070553. Процедура повторена: A на эпохе 1 остановлен посреди расчёта, B избран после истечения старой сессии и назначен эпохой 2. Измеренное переключение - 8,234 секунды. B продолжил расчёт после пятого чанка, сверил все частоты и выполнил единственную успешную публикацию указателя версии 1.

Когда A возобновился, его приложение попыталось вызвать Kazoo zk.set(pointer, payload, version=0). Но после записи B действительная версия указателя уже была равна 1. ZooKeeper отказал в условном обновлении и вернул исключение BadVersionError. Старый процесс зарегистрировал отказ, результат B не был перезаписан.

```
  zk.set(pointer_path, payload, version=expected_pointer_version)
```

Таблица 8 - Подтверждение защиты устаревшей записи

Событие	Зафиксированный результат
Пауза A	2026-10-26T07:05:59.021+00:00
Избрание B	8,234 с после паузы (host monotonic)
Продолжение B	Из чекпоинта после чанка 5
Публикация B	2026-10-26T07:06:10.534+00:00; версия 1; эпоха 2
Попытка A	2026-10-26T07:06:41.663+00:00; BadVersionError
Конечный указатель	owner=b; epoch=2; version=1
Контрольные частоты пар	EXACT MATCH: 22 311 ключей, 134 786 вхождений

Фрагмент подлинных событий журналов, укороченный до диагностических полей

```
{"worker":"b", "event":"PUBLISHED_RESULT", "epoch":2, "pointer_version":1}
{"worker":"a", "event":"BadVersionError", "epoch":1,
"expected_pointer_version":0, "reason":"STALE_EPOCH_FENCED"}
```

Важно разграничить понятия: в этом стенде защита реализована атомарным сравнением версии znode-указателя внутри ZooKeeper, а эпоха используется как дополнительный идентификатор владения. Это подлинный BadVersionError, однако он не доказывает автоматическую блокировку сторонних записей в RustFS: для отдельного внешнего получателя нужна аналогичная проверка на его стороне.

10. Сравнение результатов и времени восстановления

Таблица 9 - Итоговое сравнение двух режимов

Показатель	FENCING=0	FENCING=1
Время failover, с	9,125	8,234
Восстановление B	после чанка 5	после чанка 5
Уникальных пар	22 311	22 311
Сумма вхождений	134 786	134 786
Поэлементная сверка	EXACT MATCH	EXACT MATCH
Успешных публикаций	2	1
Устаревший A	записал результат	BadVersionError
Версия указателя	2	1
Владелец конечного указателя	A, эпоха 1 (устарел)	B, эпоха 2 (актуален)

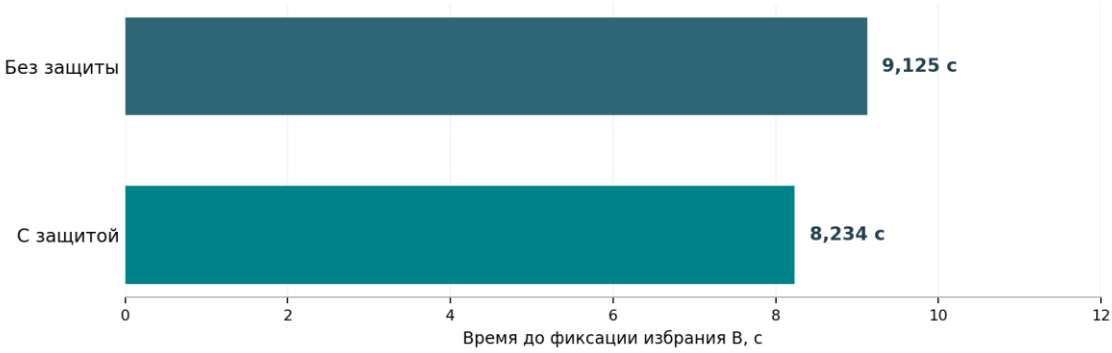


Рисунок 4 - Измеренное время передачи лидерства резервному координатору

Разница в наблюдавшихся временах составляет 0,891 секунды. На основании только двух опытов нельзя делать вывод, что включение FENCING само по себе ускоряет failover: механизм защиты применяется при публикации после расчёта, тогда как время избрания определяется сессией, состоянием ZooKeeper и моментом опроса журнала.

Итоговые CSV вычислялись независимо в каждом из четырёх завершивших работу процессов (A и B в обоих сценариях), что подтверждено событиями EXACT_MATCH. Проверка охватывала все 22 311 пар, а не только два контрольных числа.

11. Теоретические вопросы

11.1. Почему три сервера, а не два и не четыре?

ZooKeeper нужен кворум большинства, то есть больше половины голосов. Если серверов всего два, для кворума потребуются оба, поэтому отказ любого сразу оставляет систему без возможности согласовать запись. При трёх участниках кворум составляет два, и система продолжает работать после отказа одного. Четырём участникам требуется уже три голоса; они тоже могут потерять не более одного сервера. Поэтому для устойчивости к одиночному отказу три участника дают тот же эффект, что четыре, при меньшем количестве ресурсов. Для устойчивости к двум отказам потребовалось бы пять участников.

11.2. Почему fencing проверяет получатель записи, а не ZooKeeper?

ZooKeeper корректно устанавливает, чья сессия сейчас действует и кто считается лидером. Но он не выключает старый процесс физически: если процесс был заморожен, после возобновления тот может по-прежнему иметь данные, сетевые соединения и возможность отправлять запросы в базу данных или объектное хранилище. ZooKeeper не перехватывает чужие

операции записи. Поэтому сама система, которая принимает итог, должна сравнивать переданный fencing token с уже принятым максимальным значением и атомарно отвергать более старые эпохи. В нашей демонстрации такой системой является znode-указатель, а несовпадение ожидаемой версии приводит к BadVersionError.

12. Воспроизводимость и состав проекта

Таблица 10 - Основные исходники и доказательства выполнения

Файл/каталог	Назначение
lab4/compose.yaml	Три контейнера ZooKeeper и два контейнера координатора
lab4/coordinator.py	Лидерство, MapReduce пар, чекпоинты, публикация и FENCING
lab4/experiment.py	Автоматизация паузы A, перехода B, времени и восстановления
lab4/quorum_tests.py	Проверка кворума, восстановления и исчезновения ephemeral znode
lab4/zk_probe.py, zk_state.py	Проверка клиентских операций и состояния znode-указателя
lab4/offline_reference_test.py	Независимая сверка всех частот пар ДЗ № 3
lab4/evidence/ensemble/	Протоколы кворума и ephemeral znode
lab4/logs/<run>/*.txt	JSON-журналы координаторов A и B
lab4/evidence/<run>/summary.json	Машиночитаемое время failover и состояние публикации
lab4/evidence/summary_for_review.txt	Краткая сводка для защиты
lab4/results/<run>/	Чекпоинт и пересчитанные частоты пар

Для запуска требуется Windows 11 с работающим Docker Desktop, доступным ZooKeeper-образом 3.9.6 и эталонными CSV ДЗ № 3. Воспроизводящий запуск следует выполнять из каталога лабораторной, не останавливая основной контейнер RustFS.

```
cd C:\rustfs-lab\devops-hw\lab4
docker compose up -d zk1 zk2 zk3
..\.venv\Scripts\python.exe .\quorum_tests.py
..\.venv\Scripts\python.exe .\experiment.py no-fencing
..\.venv\Scripts\python.exe .\experiment.py fencing
```

Опыты сохраняют собственные каталоги результатов и не изменяют эталоны ДЗ № 3. Указатель для каждого опыта создаётся заново, благодаря чему требуемая версия 1 либо 2 не зависит от более ранних запусков.

13. Ограничения и итоговые выводы

Ансамбль из трёх серверов был развёрнут; роли leader/follower подтверждены. При потере одного последователя клиентские операции оставались успешными, при потере двух работа с кворумом прекращалась. После возврата серверов ансамбль восстановил роли. Эфемерный узел, созданный в одной клиентской сессии, исчез после её завершения.

Два координатора выполнили подлинный расчёт пар DevOps-навыков и продолжили его после приостановки A. Запасной B восстанавливался с пятого чекпоинта. Во всех четырёх итоговых расчётах получено 22 311 пар и 134 786 вхождений; частота каждого ключа совпала с ДЗ № 3.

Демонстрация без защиты показала две успешные записи и указатель версии 2, принадлежащий устаревшему A. При включённом FENCING запись возобновившегося A была отклонена ошибкой BadVersionError, а указатель версии 1 остался за B. Измеренные времена переключения: 9,125 и 8,234 секунды соответственно.